

小红书前端一面面经

小红书前端一面面经

时长:40min

1. 自我介绍
2. 大文件分片上传是怎么实现的为什么要做虚拟列表
3. 虚拟列表对性能和体验有哪些优化呢
4. webpack 和 vite 有哪些区别呢
5. 讲一下 http 不同版本的区别前端常用的请求头有哪些讲一下 tcp 的队头阻塞讲一下 tcp 的流量控制
6. token 用户登录用是怎么做的前端用户身份验证有哪些方案呢
7. csrf 攻击有考虑过吗
8. 手写题：react 写一个弹窗组件

参考答案

2. 大文件分片上传是怎么实现的？为什么要做虚拟列表？

大文件分片上传的实现：

1. **分片**：将大文件按一定大小（如 2MB）切割成多个小块（chunk）。
2. **上传**：通过 HTTP 请求逐个上传这些小块到服务器。
3. **合并**：服务器在接收到所有小块后，将它们合并成一个完整的文件。
4. **校验**：可以通过 MD5 等哈希算法校验文件的完整性。

为什么要做虚拟列表：

虚拟列表（Virtual List）通常用于处理长列表的渲染问题。在 Web 应用中，如果直接渲染一个包含大量元素的长列表，会导致性能问题，如内存占用高、渲染时间长等。虚拟列表通过只渲染当前视口（viewport）内的元素，以及根据需要动态加载和卸载其他元素，从而优化性能和用户体验。

3. 虚拟列表对性能和体验有哪些优化？

性能优化：

1. **减少 DOM 节点**：只渲染视口内的元素，减少了 DOM 节点的数量，降低了内存占用。
2. **减少重绘和重排**：由于只更新视口内的元素，减少了页面的重绘和重排次数。

体验优化：

3. **滚动流畅**：由于减少了 DOM 节点的数量，滚动操作更加流畅。
4. **快速加载**：只加载视口内的数据，减少了数据的加载时间。

4. webpack 和 vite 有哪些区别？

webpack：

1. **打包工具**：webpack 是一个模块打包工具，它将项目的所有依赖和资源打包成一个或多个 bundle。
2. **配置复杂**：webpack 的配置相对复杂，需要手动配置各种 loader 和 plugin。
3. **热更新**：支持模块热替换（HMR），但配置起来较复杂。

vite：

4. **开发服务器**：vite 是一个面向现代浏览器的构建工具和开发服务器，它利用 ES Module（ESM）的浏览器原生支持，实现了极快的冷启动和热更新。
5. **配置简单**：vite 的默认配置已经足够好，大多数情况下不需要手动配置。
6. **插件生态**：vite 的插件生态虽然不如 webpack 丰富，但正在快速发展。

5. HTTP 不同版本的区别、前端常用的请求头、TCP 的队头阻塞和流量控制

HTTP 不同版本的区别：

HTTP/1.0：每个连接只能处理一个请求，服务器处理完请求并收到客户端的应答后才会关闭连接。

HTTP/1.1：引入了持久连接（persistent connection），允许通过同一个连接发送多个请求，提高了传输效率。同时支持缓存控制、断点续传等功能。

HTTP/2：多路复用（multiplexing），允许在同一个连接上同时发送多个请求和响应，进一步提高了传输效率。头部压缩（header compression）减少了传输的数据量。

HTTP/3：基于 QUIC 协议，实现了更低的延迟和更好的多路复用。

前端常用的请求头：

Content-Type：指定请求体的媒体类型。

Authorization：包含认证信息，如 Bearer Token。

User-Agent：客户端的标识信息，包括浏览器类型、版本等。

Accept：指定客户端能够处理的媒体类型。

TCP 的队头阻塞：

TCP 的队头阻塞（Head-of-Line Blocking, HOL Blocking）是指当 TCP 连接中的一个数据包丢失时，整个连接上的后续数据包都会被阻塞，直到丢失的数据包被重传并成功接收。这会导致传输效率的降低。

TCP 的流量控制：

TCP 的流量控制通过滑动窗口（Sliding Window）机制实现。发送方和接收方各自维护一个窗口，窗口的大小表示当前可以发送或接收的数据量。当接收方的窗口满时，发送方会停止发送数据，直到接收方窗口有空间为止。这避免了发送方发送过多数据导致接收方处理不过来。

6. Token 用户登录和前端用户身份验证方案

Token 用户登录：

1. **用户输入凭证：**用户在登录页面输入用户名和密码。
2. **服务器验证：**服务器验证用户名和密码的正确性。
3. **生成 Token：**验证成功后，服务器生成一个包含用户信息的 Token（如 JWT）。
4. **返回 Token：**服务器将 Token 返回给客户端，通常通过 HTTP 响应头（如 `Authorization`）或 JSON 响应体。
5. **存储 Token：**客户端将 Token 存储在本地（如 `localStorage` 或 `cookie`），并在后续请求中携带该 Token。
6. **验证 Token：**服务器在接收到请求后，验证 Token 的有效性，并根据 Token 中的信息处理请求。

前端用户身份验证方案：

7. **Session-Based Authentication：**基于会话的身份验证，服务器在验证用户后创建一个会话，并在客户端存储会话 ID（如 `cookie`）。

8. **Token-Based Authentication**: 基于 Token 的身份验证, 如 JWT (JSON Web Token)。
9. **OAuth**: 开放授权协议, 允许用户授权第三方应用访问其资源 (如社交媒体账号)。
10. **OpenID Connect**: 基于 OAuth 的认证协议, 用于身份验证和信息交换。

7. CSRF 攻击有考虑过吗?

CSRF (Cross-Site Request Forgery) 攻击:

CSRF 攻击是指攻击者诱导用户在已登录的网站上执行非预期的操作 (如转账、发布内容等)。由于用户已经登录, 浏览器会自动携带认证信息 (如 cookie), 导致攻击成功。

防御措施:

1. **检查 Referer 头**: 服务器在接收到请求时, 检查 Referer 头是否来自受信任的源。但这种方法存在被绕过的风险。
2. **使用 CSRF Token**: 在表单或请求中添加一个随机的 CSRF Token, 服务器在接收到请求时验证该 Token 的有效性。
3. **SameSite Cookie 属性**: 设置 cookie 的 SameSite 属性为 `Strict` 或 `Lax`, 限制 cookie 的跨站发送。

8. 手写题: React 写一个弹窗组件

Plain Text

```
1 import React, { useState } from 'react';
2 import './Modal.css'; // 假设有一个CSS文件用于样式
3
4 const Modal = ({ isOpen, onClose, children }) => {
5   if (!isOpen) {
6     return null;
7   }
8
9   return (
10     <div className="modal-overlay" onClick={onClose}>
11       <div className="modal-content" onClick={(e) => e.stopPropagation
12         ()}>
13         <div className="modal-header">
14           <span className="close-button" onClick={onClose}>&times;</span>
15         </div>
16         <div className="modal-body">
17           {children}
18         </div>
19       </div>
20     </div>
21   );
22
23 export default Modal;
24
25 // Modal.css
26 /*
27 .modal-overlay {
28   position: fixed;
29   top: 0;
30   left: 0;
31   right: 0;
32   bottom: 0;
33   background: rgba(0, 0, 0, 0.5);
34   display: flex;
35   justify-content: center;
36   align-items: center;
```

```
37   z-index: 1000;
38 }
39
40 .modal-content {
41   background: white;
42   padding: 20px;
43   border-radius: 5px;
44   position: relative;
45 }
46
47 .modal-header {
48   display: flex;
49   justify-content: space-between;
50   align-items: center;
51   margin-bottom: 10px;
52 }
53
54 .close-button {
55   cursor: pointer;
56   font-size: 20px;
57 }
58
59 .modal-body {
60   /* 根据需要添加样式 */
61 }
62 */
63
64 // 使用示例
65 const App = () => {
66   const [isModalOpen, setIsModalOpen] = useState(false);
67
68   return (
69     <div>
70       <button onClick={() => setIsModalOpen(true)}>Open Modal</button>
71       <Modal
72         isOpen={isModalOpen}
73         onClose={() => setIsModalOpen(false)}
74       >
75         <h2>Modal Title</h2>
76         <p>This is a modal content.</p>
```

```
77     </Modal>
78   </div>
79 );
80 };
81
82 export default App;
```

这个简单的 React 弹窗组件 `Modal` 接受三个 props: `isOpen` (控制弹窗的显示与隐藏)、`onClose` (关闭弹窗时的回调函数)、`children` (弹窗的内容)。CSS 样式用于美化弹窗的外观。在 `App` 组件中, 通过点击按钮来控制弹窗的显示与隐藏。